Nigehban AI (Crisis Intelligence System)

Complete Project Documentation

1. Executive Summary

Nigehban AI (formerly AegisNet) is an advanced, fully autonomous, multi-agent artificial intelligence platform designed to track, analyze, and map national crises across Pakistan in real time.

The system is engineered as a high-fidelity "Command Center" dashboard that requires zero human intervention to gather data. It continuously ingests machine-readable intelligence from global APIs, validates the threat using a centralized AI reasoning engine, and pushes actionable alerts to a futuristic glassmorphic HUD.

2. Architecture & Workflows

The architecture is split into two primary decoupled components:

A. The Reasoning Engine (Spring Boot Backend)

Running on Java 21, the backend acts as the autonomous brain. It relies on concurrent execution threads driven by the `@Scheduled` annotation. - **`AutonomousMonitorService`**: The master orchestrator. It manages 5 distinct polling cycles (ranging from every 30 seconds to every 5 minutes) to ensure rate limits are respected while keeping data fresh. It broadcasts all processed intelligence via **WebSocket (STOMP/SockJS)** directly to the connected clients. - **`CrisisIntelligenceAgent`**: The core reasoning module. This agent takes raw data (e.g., a "temperature: 46C" reading or a news article titled "Chemical Spill in Lahore") and maps it against a highly structured national hazard taxonomy. It automatically estimates affected populations, calculates a Criticality Score (0-100), and generates recommended response actions (e.g., "Deploy Military CBRN units").

B. The Tactical HUD (Angular Frontend)

Running on Angular 17, the frontend is a strictly real-time visualization layer. - It subscribes to `/topic/city-threats` and `/topic/crisis-events`. - Features an animated satellite map with real-time tracking of five primary sectors (Karachi, Lahore, Islamabad, Peshawar, Quetta) alongside broader regional tracking (Gilgit, Swat Valley, AJK). - Employs rich, dynamic CSS animations (scanlines, pulsing threats, glassmorphism) to highlight severe alerts.

3. Agent-Driven Data Integration Pipelines

To ensure comprehensive situational awareness, Nigehban AI uses a 5-Agent orchestration pipeline:

- **Agent 1 (Weather & Environment)**: Hits the **Open-Meteo API** every 30s. Tracks extreme temperatures (Heatwaves), flash flooding (River Discharges > 1000m³/s), severe winds (Cyclones), and air quality (AQI tracking for Smog).
- **Agent 3 (News Intelligence)**: Polls the **GDELT Project** every 90s. Scans global and local media for semantic signs of infrastructure collapse, terrorism, or civic disruption. If the live feed is sparse, it falls back to a high-fidelity historical dataset to simulate crises like Dam Failures or Glacial Lake Outburst Floods (GLOFs).
- **Agent 4 (Decentralized Social)**: Scrapes **Bluesky (AT-Protocol)** and **Mastodon** every 60s. Uses keyword-velocity analysis to track rapidly escalating on-the-ground situations (e.g., Riots, Emergency incidents) before official news covers them.
- **Agent 5 (Advanced Intel)**: Checks **NASA FIRMS** (for thermal anomalies and forest fires), **HDX** (for humanitarian/drought datasets), and **PMD** (Pakistan Meteorological Department official CAP RSS advisories) every 2 minutes.
- **Agent 6 (Global Disasters)**: Pulls the **GDACS (EU Joint Research Centre)** GeoJSON feed every 5 minutes to track major tectonic or meteorological shifts.

4. Hazard Taxonomy

The system is explicitly programmed to recognize, categorize, and respond to: 1. **Hydrological / Meteorological**: Urban inundation, riverine floods, severe cyclones, heatwaves, freezing snowstorms, and droughts. 2. **Geological**: Earthquakes, avalanches, landslides. 3. **Environmental / Human-Induced**: Forest fires, chemical spills (HAZMAT), dam structural failures, power grid collapses. 4. **Biological / Security**: Mass civil unrest (Riot), mass casualty incidents (MCI), epidemics, and agricultural plagues (locusts).

5. Deployment Instructions

Requirements: JDK 21+, Node.js v18+, Maven.

Step 1: Start the Core Engine (Backend) 1. Navigate to the `/backend` directory. 2. Set your `JAVA_HOME` to point to a valid JDK 21 installation. 3. Run the command: `mvn spring-boot:run` 4. The Tomcat server will start on **port 8080**, and the Autonomous Monitor will immediately begin polling.

Step 2: Start the Tactical HUD (Frontend) 1. Navigate to the `/frontend` directory. 2. Run `npm install` (if dependencies are not already installed). 3. Run `npm start` (or `ng serve`). 4. Access the dashboard at **`http://localhost:4200`**. The HUD will instantly connect to the backend WebSocket and render live data.